CHAPTER 1

DESIGN TYPEAHEAD SUGGESTION

1 System Design: The Typeahead Suggestion System 2
2 Requirements of the Typeahead Suggestion System's Design 3
3 High-Level Design of the Typeahead Suggestion System 6
4 Data Structure for Storing Prefixes . 7
5 Detailed Design of the Typeahead Suggestion System 9
6 Evaluation of the Typeahead Suggestion System's Design 11
7 Quiz on the Typeahead Suggestion System's Design 13

Discover the typeahead suggestion System Design, covering aspects like efficient data structures and assemblers for updating suggestions.

1 System Design: The Typeahead Suggestion System

1.1 Introduction

Typeahead suggestion, also referred to as the **autocomplete system**, is a front-end functionality that provides real-time search query suggestions as users type. It enables users to search for a known and frequently searched query. This feature comes into play when a user types a query in the search box. The typeahead system provides a list of top suggestions to complete a query based on the user's search history, the current context of the search, and trending content across different users and regions. Frequently searched queries always appear at the top of the suggestion list. The typeahead system doesn't search faster. However, it helps the user form a sentence more quickly. The underlying algorithm determines the relevance and ranking of suggested terms.

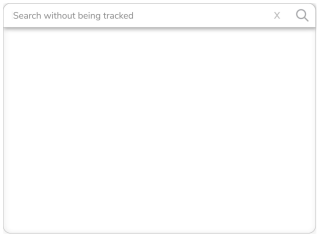

Figure 1.1: The typeahead suggestion system in action

Typeahead systems are common in applications like:

- Search engines such as Google or Bing
- E-commerce websites to autocomplete product names
- Text and code editors

Note: To deliver a seamless user experience, the system must have low latency and high fault tolerance for typeahead suggestions. The typeahead suggestion systems designed by tech giants, such as Google and Amazon, enhance search accuracy and personalize the user experience by analyzing past interactions and preferences.

1.2 Typeahead design problem

As system designers or a candidate for system design interviews, we should be aware of the workings of the typeahead suggestion system. There could be interesting system design interview questions that an interviewer can ask:

- How would you design a typeahead suggestion system that is capable of learning from user behavior—how would you incorporate real-time data processing and feedback loops?
- How does load balancer placement and configuration impact the typeahead system performance and user experience?
- How would you efficiently update sub-tree structures in a typeahead system to accommodate new data ingestion from various sources, such as web crawlers, etc.?

1.3 A glimpse from the chapter

A robust typeahead system requires careful consideration of several architectural components. For example, key-value stores often support the backend, providing fast data retrieval. Similarly , partitioning the data across multiple application servers enhances scalability and performance. To improve suggestion quality, the offline map-reduce jobs can be used to preprocess and analyze vast amounts of data.

To achieve high availability and fault tolerance of the service, replication of data across servers is necessary. Furthermore , real-time metrics on system performance, such as latency and throughput, are essential for identifying bottlenecks and optimizing the system. Carefully designed interfaces between the frontend and backend, along with effective load balancing, contribute to a seamless user experience across diverse use cases. Let's move on to the plan to design a typeahead suggestion system.

1.4 How will we design a typeahead suggestion system?

We've divided the chapter on the design of the typeahead suggestion system into six lessons:

1. **Requirements:** In this lesson, we focus on the functional and non-functional requirements for designing a typeahead suggestion system. We also discuss resource estimations for the smooth operation of the system.

2. **High-level design:** In this lesson, we discuss the high-level design of our version of the typeahead suggestion system. We also discuss some essential APIs used in the design.

3. **Data structure for storing prefixes:** In this lesson, we cover the trie data structure (an efficient tree data structure) that's used to store search prefixes. We also discuss how it can be further optimized to reduce the tree traversal time.

4. **Detailed design:** In this lesson, we explain the two main components, the *suggestions* service and the *assembler*, which make up the detailed design of the typeahead suggestion system.

5. **Evaluation:** This lesson evaluates the proposed design of the typeahead suggestion system based on the non-functional requirements of the system. It also presents some client-side optimization and personalization that could significantly affect the system's design.

6. **Quiz:** In this lesson, we assess your understanding of the design via a quiz.

If you want to gain hands-on experience with building a basic typeahead suggestion system in AWS, consider exploring the following AWS Cloud Lab.

Let's start by identifying the requirements for designing a typeahead suggestion system.

2 Requirements of the Typeahead Suggestion System's Design

2.1 Requirements

In this lesson, we look into the requirements and estimated resources that are necessary for the design of the typeahead suggestion system. Our proposed design should meet the following requirements.

Functional requirements

The system should suggest top N (let's say top ten) frequent and relevant terms to the user based on the text a user types in the search box.

Non-functional requirements

- **Low latency:** The system should show all the suggested queries in real time after a user types. The latency shouldn't exceed 200 ms. A study suggests that the average time between two keystrokes is 160 milliseconds. So, our time budget of suggestions should be greater than 160 ms to give a real-time response. This is because if a user is typing fast, they already know what to search and might not need suggestions. At the same time, our system response should be greater than 160 ms. However, it should not be too high because in that case, a suggestion might be stale and less useful.

- **Fault tolerance:** The system should be reliable enough to provide suggestions despite the failure of one or more of its components.
- **Scalability:** The system should support the ever-increasing number of users over time.

2.2 Resource estimation

As was stated earlier, the typeahead feature is used to enhance the user experience while typing a query. We need to design a system that works on a scale that's similar to Google Search. Google receives more than 3.5 billion searches every day. Designing such an enormous system is a challenging task that requires different resources. Let 's estimate the storage and bandwidth requirements for the proposed system.

Storage estimation

Assuming that out of the 3.5 billion queries per day, two billion queries are unique and need to be stored. Let 's also assume that each query consists of 15 characters on average, and each character takes 2 Bytes of storage. According to this formulation, we would require the following: 2 billion $\times$ 15 $\times$ 2 = 60GB to store all the queries made in a day.
Storage required per year: 60 GB/day $\times$ 365 = 21.9 TB/year

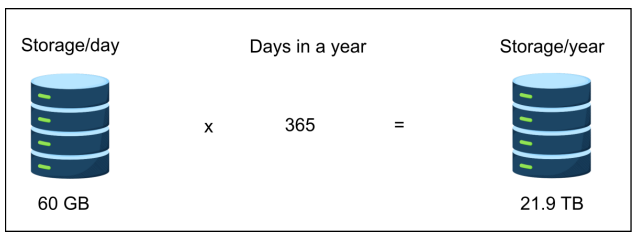

Figure 1.2: Storage requirements for the typeahead suggestion system

Bandwidth estimation

3.5 billion queries will reach our system every day. Assume that each query a user types is 15 characters long on average.
Keeping this in mind, the total number of reading requests of characters per day would be as follows:
15 $\times$ 3.5 billion = 52.5 billion characters per day.
Total read requests per second: 52.5B/86400 $\approx$ 0.607M characters/sec. 86,400 is the number of seconds per day.
Since each character takes 2 Bytes, the bandwidth our system would need is as follows:
0.607 million $\times$ 2 $\times$ 8 = 9.7 Mb/sec
9.7 Mb/sec is the incoming bandwidth requirement for queries that have a maximum length of 15 characters. Our system would suggest the top ten queries that are roughly of the same length as the query length after each character a user types. Therefore, the outgoing bandwidth requirement would become the following:

$$10 \times 9.7 \text{ Mb/sec} = 97 \text{ Mb/sec}$$

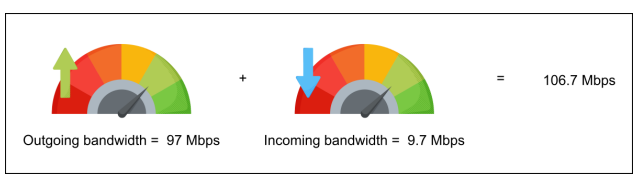

Figure 1.3: The total bandwidth required by the typeahead suggestion system

Number of servers estimation

Our system will receive 52.5 billion requests per day. Considering our assumption of using daily active users as a proxy for the number of requests per second for peak load, we get 52.5 billion requests per second (considering each character as a request).
Recall that a typical server can serve 64,000 requests per second (RPS). So, the required servers can be estimated using the following formula:

$$\text{Number of servers} = \frac{\text{Number of requests/second}}{\text{RPS of server}}$$

$$\text{Number of servers} = \frac{52.5 \text{ billion}}{64,000} = 820312.5 \approx 821 \text{ K servers}$$

> The initial estimate of 52.5 billion requests per second seems unrealistic. However, on average, a person types at a rate of around three to four characters per second.

Therefore, assuming 3.5 billion users and a peak load scenario where each user types roughly three characters per second, we arrive at a more feasible estimate of (3.5×3) billion requests per second. This revised figure provides a more realistic estimation as follows:

$$\text{Number of Servers} = \frac{3.5 \text{ billion} \times 3}{64,000} = 164062.5 \approx 164 \text{ K servers}$$

Figure 1.4: The number of servers required for the typeahead suggestion system

In the table below, adjust the values to see how the resource estimations change.

2.3 Building blocks we will use

The design of the typeahead suggestion system consists of the following building blocks that have been discussed in the initial chapters of the course:

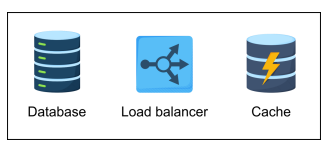

Figure 1.5: Building blocks required in the design of the typeahead suggestion system

- **Databases** are required to keep the data related to the queries' prefixes.
- **Load balancers** are required to disseminate incoming queries among a number of active servers.
- **Caches** are used to keep the top N suggestions for fast retrieval.

In the next lesson, we'll focus on the high-level design and APIs of the typeahead suggestion system.

3 High-Level Design of the Typeahead Suggestion System

3.1 High-level design

According to our requirements, the system shouldn't just suggest queries in real time with minimum latency but should also store the new search queries in the database. This way, the user gets suggestions based on popular and recent searches.
Our proposed system should do the following:

- Provide suggestions based on the search history of the user.
- Store all the new and trending queries in the database to include them in the list of suggestions.

When a user starts typing a query, every typed character hits one of the application servers. Let's assume that we have a **suggestions service** that obtains the top ten suggestions from the cache, Redis, and returns them as a response to the client. In addition to this, suppose we have another service known as an **assembler**. An assembler collects the user searches, applies some analytics to rank the searches, and stores them in a NoSQL database that's distributed across several nodes.

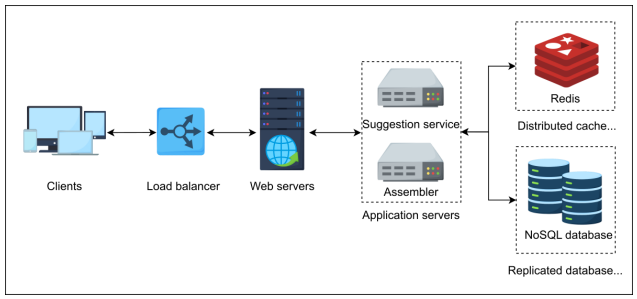

Figure 1.6: The high-level design of the typeahead suggestion system

Furthermore, we also need load balancers to distribute the incoming requests evenly. We also add web servers as entry points for clients so that they can forward requests to the appropriate microservices. These web servers encapsulate the internal system architecture and provide other services, such as authentication, monitoring, request shaping, management, and more.

3.2 API design

Since the system provides suggestions to the user and adds trending queries to the databases, we need the following APIs.

Get suggestion

```
getSuggestions(prefix)
```

This API call retrieves suggestions from the servers. This call is made via the suggestion service and returns the response to the client.
The following table explains the parameter that's passed to the API call:

Parameter	Description
`prefix`	This refers to whatever the user has typed in the search bar.

Add trending queries to the database

```
addToDatabase(query)
```

This API call adds a trending `query` to the database via an assembler if the query has already been searched and has crossed a certain threshold.

Parameter	Description
`query`	This represents a frequently searched query that crosses the predefined limit.

> **Quiz: Question 1:** Instead of updating the whole page, we just need to update the suggested query in the search box in real time. What technique can we use for this purpose? **Answer:** We should update the query asynchronously using AJAX. AJAX is a set of web development techniques. It allows web pages to exchange a small amount of data, as in the typeahead suggestion system, with the server without interfering with the display and behaviors of the existing web page.

Suppose our typeahead system needs to support multiple languages simultaneously to cater to a global user base. What are the potential bottlenecks in the current high-level design, and how will we solve them? To get started, say "hello" to Ed in the widget below, and it will lead the way.

In the next lesson, we'll learn about an efficient data structure that's used to store the suggestions or prefixes.

4 Data Structure for Storing Prefixes

4.1 The trie data structure

Before we move on to the discussion of the detailed design of the typeahead suggestion system, we must choose an efficient data structure to store the prefixes. Prefixes are the groups of characters a user types. The issue we're attempting to tackle is that we have many `strings` that we need to store in a way that allows users to search for them using any prefix. Our service suggests the next words that match the provided prefix. Let's suppose our database contains the phrases `UNITED`, `UNIQUE`, `UNIVERSAL`, and `UNIVERSITY`. Our system should suggest "**UNIVERSAL**" and "**UNIVERSITY**" when the user types "**UNIV**."

There should be a method that can efficiently store our data and help us conduct fast searches because we have to handle a lot of requests with minimal latency. We can't rely on a database for this because providing suggestions from the database takes longer as compared to reading suggestions from the RAM(Recall from the Back-of-the-Envelope Calculation chapter that reading 1MB of data sequentially from RAM takes 9 microseconds while from persistent storage like SSD takes 200 microseconds.). Therefore, we need to store our index in memory in an efficient data structure. However, for durability and availability, this data is stored in the database.

The **trie** (pronounced "try") is one of the data structures that's best suited to our needs. A **trie** is a tree-like data structure for storing phrases, with each tree node storing a character in the phrase in order. If we needed to store `UNITED`, `UNIQUE`, `UNIVERSAL`, and `UNIVERSITY` in the trie, it would look like this:

Component type 'NaryTree' not yet supported.

If the user types "UNIV," our service can traverse the trie to go to the node `V` to find all the terms that start with this prefix—for example, `UNIVERSAL`, `UNIVERSITY`, and so on.

The trie can combine nodes as one where only a single branch exists, which reduces the depth of the tree. This also reduces the traversal time, which in turn increases the efficiency. As an example, a space- and time-efficient model of the above trie is the following:

Component type 'NaryTree' not yet supported.

Track the top searches

Since our system keeps track of the top searches and returns the top suggestion, we store the number of times each term is searched in the trie node. Let 's say that a user searches for `UNITED` 15 times, `UNIQUE` 20 times, `UNIVERSAL` 21 times, and `UNIVERSITY` 25 times. In order to provide the top suggestions to the user, these counts are stored in each node where these terms terminate. The resultant trie looks like this:

Component type 'NaryTree' not yet supported.

If a user types "UNI," the system starts traversing the tree under the root node for `UNI`. After comparing all the terms originating from the root node, the system provides suggestions of all the possible words. Since the frequency of the word `UNIVERSITY` is high, it appears at the top. Similarly , the frequency of the word `UNITED` is relatively low, so it appears last. If the user picks `UNIQUE` from the list of suggestions, the number against `UNIQUE` increases to 21.

> **Quiz: Question 1:** We reduced the time to traverse the trie by combining nodes with single branches and reducing the number of levels. Is there any other way to minimize the trie traversal time? **Answer:** One way to reduce the trie traversal time is to #key# pre-compute:
>
> Instead of traversing the tree each time, the system traverses the tree only once and stores the suggestions in each node which we call pre-computed suggestions. #key# and save the top ten (or any number of our choosing) suggestions for every prefix in the node. This means that instead of traversing the trie each time a user types in "UNIVERS" into the search box, the system will have precomputed, sorted, and stored the solution to the prefix 'UNIVERS'—that is, 'UNIVERSITY', 'UNIVERSAL', and so on—inside the node that carries the prefix 'UNIVERS'. However, this approach requires extra space to save precomputed results.

Trie partitioning

We aim to design a system like Google that we can use to handle billions of queries every second. One server isn't sufficient to handle such an enormous amount of requests. In addition to this, storing all the prefixes in a single trie isn't a viable option for the system's availability, scalability, and durability. A good solution is to split the trie into multiple tries for a better user experience.

Let's assume that the trie is split into two parts, and each part has a replica for durability purposes. All the prefixes starting from "A" to "M" are stored on Server/01, and the replica is stored on Server/02. Similarly , all the prefixes starting from "N" to "Z" are stored on Server/03, and the replica is stored on Server/04. It should be noted that this simple technique doesn't always balance the load equally because some prefixes have many more words while others have fewer. We use this simple technique to understand partitioning.

We can split the trie into as many parts as we wish to distribute the load on to different servers and achieve the desired performance.

Partitioned Trie

Prefixes	Primary	Secondary
A to M	Server/01	Server/02
N to Z	Server/03	Server/04

Table 1.1: Partitioned Trie

> **Quiz: Question 1:** Where will the mapping between the prefixes and their primary and secondary storage be stored? Who will manage and direct the requests to these servers? **Answer:** In a distributed system where multiple clusters consisting of several servers can be used for a specific service, we use a cluster manager like ZooKeeper to store the mapping between clusters.

Process a query after partitioning When a user types a query, it hits the load balancer and is forwarded to one of the application servers. The application server searches the appropriate trie depending on the prefix typed by the user. For example, if a user types something starting from "U," it either accesses Server/03 or Server/04 since both have the tries stored on them that have prefixes starting with "U."

Update the trie

Billions of searches every day give us hundreds of thousands of queries per second. Therefore, the process of updating a trie for every query is highly resource intensive and time-consuming and could hamper our read requests. This issue can be resolved by updating the trie offline after a specific interval. To update the trie offline, we log the queries and their frequency in a hash table and aggregate the data at regular intervals. After a specific amount of time, the trie is updated with the aggregated information. After the update of the trie, all the previous entries are deleted from the hash table.

Prefixes and Their Frequencies Updated Periodically

Prefix	Time Interval (One Hour)	Frequency
UNIVERSITY	1st hour	25
UNIVERSITY	2nd hour	60
UNIVERSITY	3rd hour	100

Table 1.2: Prefixes and Their Frequencies Updated Periodically

We can put up a **MapReduce (MR)**(MapReduce is a big data processing engine.) job to process all of the logging data regularly, let's say every 15 minutes. These MR services calculate the frequency of all the searched phrases in the previous 15 minutes and dump the results into a hash table in a database like Cassandra. After that, we may further update the trie with the new data. We can update the current copy of the trie with all of the new words and their frequencies. We should perform this offline because our priority is to provide suggestions to users instead of keeping them waiting.
Primarily, we can update the trie using the following two approaches.

- We can replicate the trie on each server to update it offline. After that, we can start using it for suggestions and throw away the old ones.
- Another way is to have one primary copy and several secondary copies of the trie. While the main copy is used to answer the queries, we may update the secondary copy. We may also make the secondary our main copy once the upgrade is complete. We can then upgrade our previous primary, which will then be able to serve the traffic as well.

  Quiz: Question 1: If the prefix frequencies keep increasing over time, the corresponding integers storing them can overflow. How can we manage this issue? **Answer:** We can normalize frequencies by mapping them in a range, let's say between zero and 1,000. Alternatively, we can stop any further additions after a certain threshold is reached, assuming that any prefix reaching that threshold is at the top of the rankings.

In the next lesson, we'll discuss the detailed design of the typeahead suggestion system.

5 Detailed Design of the Typeahead Suggestion System

5.1 Detailed design

Let's go over the flow and interaction of the components shown in the illustration below. Our design is divided into two main parts:

- A suggestion service
- An assembler

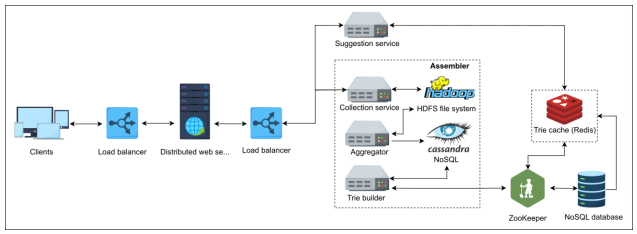

Figure 1.7: The detailed design of the typeahead suggestion system

Suggestion service

At the same time that a user types a query in the search box, the `getSuggestions(prefix)` API calls hit the suggestions services. The top ten popular queries are returned from the distributed cache, Redis.

Assembler

In the previous lesson, we discussed how tries are built, partitioned, and stored in the database. However, the creation and updation of a trie shouldn't come in the critical path of a user's query. We shouldn't update the trie in real time for the following reasons:

- There could be millions of users entering queries every second. During such phases with large amounts of incoming traffic, updating the trie in real time on every query can slow down our **suggestion service**.
- We have to provide top suggestions that might not frequently change after the creation or updation of the trie. So , it's less important to update the trie frequently.

In light of the reasons given above, we have a separate service called an assembler that's responsible for creating and updating tries after a certain configurable amount of time. The assembler consists of the following different services:

- **Collection service:** Whenever a user types, this service collects the log that consists of phrases, time, and other metadata and dumps it in a database that's processed later. Since the size of this data is huge, the **Hadoop Distributed File System (HDFS)** is considered a suitable storage system for storing this raw data. An example of the raw data from the collection service is shown in the following table. We record the time so that the system knows when to update the frequency of a certain phrase.

Raw Data Collected by the Collection Service

Phrases	Date and Time (DD-MM-YYYY HH: MM: SS)
UNIVERSAL	23-03-2022 10:16:18
UNIVERSITY	23-03-2022 10:20:11
UNIQUE	23-03-2022 10:21:10
UNIQUE	23-03-2022 10:22:24
UNIVERSITY	23-03-2022 10:25:09

Table 1.3: Raw Data Collected by the Collection Service

- **Aggregator:** The raw data collected by the **collection service** is usually not in a consolidated shape. We need to consolidate the raw data to process it further and to create or update the tries. An aggregator retrieves the data from the HDFS and distributes it to different workers. Generally, the MapReducer is responsible for aggregating the frequency of the prefixes over a given interval of time, and the frequency is updated periodically in the associated Cassandra database. **Cassandra** is suitable for this purpose because it can store large amounts of data in a tabular format. The following table shows the processed and consolidated data within a particular period. This table is updated regularly by the aggregator and is stored in a hash table in a database like Cassandra. For simplicity, we assume that our data is case insensitive.

Useful Information Extracted from the Raw Data

Phrases	Frequency	Time Interval
UNIVERSAL	1400	1st 15 minutes
UNIVERSITY	1340	1st 15 minutes
UNIQUE	1200	1st 15 minutes

Table 1.4: Useful Information Extracted from the Raw Data

- **Trie builder:** This service is responsible for creating or updating tries. It stores these new and updated tries on their respective shards in the trie database via ZooKeeper. Tries are stored in persistent storage in a file so that we can rebuild our trie easily if necessary. NoSQL document databases such as MongoDB are suitable for storing these tries. This storage of a trie is needed when a machine restarts. The trie is updated from the aggregated data in the Cassandra database. The existing snapshot of a trie is updated with all the new terms and their corresponding frequencies. Otherwise, a new trie is created using the data in the Cassandra database. Once a trie is created or updated, the system makes it available for the suggestion service.

Let's assess your understanding by responding to the question below:
How would you handle misspelled or incomplete words in the Typeahead Suggestion System?

> **Quiz: Question 1:** Should we collect data and build a trie per user, or should it be shared among all users? **Answer:** Since we aim to design a system on a scale that's similar to Google
>
> Search, billions of users will be using our service. Since the number of users would be huge, maintaining a tree for each user would become complex and time-consuming. There's also a possibility of duplicated trees if several users have typed some common searches, resulting in resource wastage. Therefore, our design assumes a common trie that's shared among users, where the ranking would be based on single phrases in the trie and the frequency of the terms.

6 Evaluation of the Typeahead Suggestion System's Design

6.1 Requirements compliance

The non-functional requirements of the proposed typeahead suggestion system are low latency, fault tolerance, and scalability.

- **Low latency:** There are various levels at which we can minimize the system's latency. We can minimize the latency with the following options:
  - Reduce the depth of the tree, which reduces the overall traversal time.
  - Update the trie offline, which means that the time taken by the update operation isn't on the clients' critical path.
  - Use geographically distributed applications and database servers. This way, the service is provided near the user, which also reduces any communication delays and aids in reducing latency.
  - Use Redis as a caching layer on top of NoSQL database clusters and Cassandra database to ensure low-latency data retrieval and improved performance.
  - Appropriately partition tries, which leads to a proper distribution of the load and results in better performance.

- **Fault tolerance:** Since the replication and partitioning of the trees are provided, the system operates with high resilience. If one server fails, others are on standby to deliver the services.
- **Scalability:** Since our proposed system is flexible, more servers can be added or removed as the load increases. For example, if the number of queries increases, the number of partitions or shards of the trees is increased accordingly.

Non-functional Requirements Compliance

Non-functional Requirements	Approaches
Low latency	• Reducing the depth of the tries makes the traversal faster • Updating the tries offline and not in real time) • Partitioning of the tries • Caching servers
Fault tolerance	• Replicating the tries and the NoSQL databases
Scalability	• Adding or removing application servers based on the incoming traffic • Increasing the trie partitions

Table 1.5: Non-functional Requirements Compliance

6.2 Client-side optimisation

To improve the user's experience, we can implement the following client-side optimizations:

- The client should only attempt to contact the server if the user hasn't pressed any keys for some time—for example, any delay greater than 160 ms, which is the average delay between two keystrokes. This way, we can also avoid unnecessary bandwidth consumption. This suggestion might not be useful when a user is typing rapidly.
- The client can initially wait till the user types a few characters.
- Clients can save a local copy of the recent history of suggestions. The rate of reuse of recent history in the suggestions list is relatively high.
- One of the most crucial elements is establishing a connection with the server as soon as possible. The client can establish a connection with the server as soon as the user visits the search page. As a result, the client doesn't waste time establishing the connection when the user inputs the first character. Usually, the connection is established with the server via a **WebSocket protocol**.
- For efficiency, the server can push a portion of its cache to CDNs and other edge caches at Internet exchange points (IXPs) or even inside a client's Internet service provider (ISP).

6.3 Personalization

Users receive typeahead suggestions based on their previous searches, location, language, and other factors. We can save each user's personal history on the server separately and cache it on the client. Before transmitting the final set to the user, the server might include these customized phrases. Personalized searches should always take precedence over other types of searches.

6.4 Summary

In this design problem, we learned how pushing resource-intensive processing to the offline infrastructure and using appropriate data structures enables us to serve our customers with low latency. Many optimizations lend themselves to specific use cases. We saw multiple optimizations on our trie data structures for condensed data storage and quick serving.

7 Quiz on the Typeahead Suggestion System's Design

Quiz

Question 1: Which component is responsible for updating the trie offline in the design of the typeahead suggestion system?

- **[CORRECT]** The assembler
- The suggestion service
- ZooKeeper
- None of the above

Question 2: What is the purpose of MapRreduce in the design of the typeahead suggestion system?

- To disseminate data toward a database for storage
- **[CORRECT]** To aggregate the prefix count
- To increase the number of required operations for optimization
- None of the above

Question 3: Why do we aggregate the counts and update them offline in their respective shards?

- To enhance the write operations
- To provide accurate results to the user
- **[CORRECT]** To minimize the latency of the read operations
- None of the above

Question 4: (Select all that apply.) Why is the trie partitioned and stored on several servers? *(Select all that apply.)*

- **[CORRECT]** To reduce the load on a single server
- To store more data
- **[CORRECT]** To increase the performance of the system

- None of the above

Question 5: (Select all that apply.) The replication of the trie partition is required to enhance _____.
(Select all that apply.)

- **[CORRECT]** fault tolerance
- **[CORRECT]** the durability of the data
- the security of the data